Programming Assignment 2

Data Prefetching and Cache Replacement in ChampSim

Team TeamName

Advay Bapat	23B1225
Aniket Gupta	23B1245
Apoorv Goyal	23B1266
Kundan Maiti	23B1242

Department of Electrical Engineering
Indian Institute of Technology Bombay

CS683: Advanced Computer Architecture

Date: October 15, 2026

Contents

1 Overview and Methodology . . . 2
1.1 Simulation Setup . . . 2
1.2 Configurations Evaluated . . . 2
1.3 Metrics . . . 3
2 Task 1: Data Prefetching . . . 3
2.1 Task 1A: Understanding the IP-Stride Prefetcher . . . 3
2.2 Task 1B: Stride Pattern Prefetcher (SPP) . . . 4
2.2.1 Final Design (spp16) . . . 4
2.2.2 How the Final Design Was Reached . . . 5
2.2.3 Accuracy-Based Throttling . . . 7
2.2.4 Storage Overhead . . . 8
2.2.5 Results . . . 8
2.2.6 Discussion . . . 8
2.2.7 Full Comparison of All Versions . . . 12
3 Task 2: Cache Replacement Policy . . . 13
3.1 Baseline: SRRIP at the L2 . . . 13
3.2 Task 2A: Instruction-Conscious RRIP (I-RRIP) . . . 13
3.2.1 Interface Change . . . 13
3.2.2 Policy Design . . . 13
3.2.3 Results . . . 13
3.3 Task 2B: Dynamic I-RRIP . . . 14
3.3.1 Set-Dueling Design . . . 14
3.3.2 Rationale for the Leader Sets and Decision Logic . . . 14
3.3.3 Sensitivity Study . . . 15
3.3.4 Results . . . 15
4 Task 3: SPP with Prefetch-Aware Dynamic I-RRIP . . . 16
4.1 Design . . . 16
4.2 Rationale for Handling Prefetched Lines . . . 16
4.3 Results . . . 16
5 Summary . . . 18
A Reproducing the Results . . . 19

1. Overview and Methodology

Task 1 replaces the provided IP-stride L1D prefetcher with a Stride Pattern Prefetcher (SPP) that keeps, for every load IP, a *short* (one-stride) and a *long* (three-stride signature) predictor, chooses between them per IP with a tournament counter, walks the predicted stride path up to eight predicted lines ahead inside the 4 KB page, and throttles its degree from measured accuracy. The final design (`spp16`, 7.8 kB of state) reaches a geometric-mean speedup of **1.0129** over the no-prefetch baseline, **5.4× the gain of IP-stride** (1.0024). *[Add one sentence each for Tasks 2 and 3 once they are done.]*

1.1 Simulation Setup

All experiments use the ChampSim simulator from the PA2 repository, with 25M warm-up instructions followed by 25M simulated instructions on each of the three provided traces (`Trace1`, `Trace2`, `Trace3`). Table 1 lists the simulated system (values from `inc/cache.h`).

Table 1. Simulated single-core system.

Component	Configuration
Branch predictor	Hashed perceptron
L1I cache	32 KB, 64 sets × 8 ways, 4-cycle latency
L1D cache	48 KB, 64 sets × 12 ways, 5-cycle latency, 16 MSHRs, 16-entry PQ
L2 cache	512 KB, 1024 sets × 8 ways, 10-cycle latency, 32 MSHRs
LLC	2 MB, 2048 sets × 16 ways, 20-cycle latency
DRAM	4 GB, 1 channel, 64-bit, 6400 MT/s
Block / page size	64 B / 4 KB
Warm-up / simulation	25M / 25M instructions

1.2 Configurations Evaluated

Table 2. Binaries built for this report (`./build_champsim.sh <L1D prefetcher> <L2C replacement> <name>`).

Name	L1D prefetcher	L2C replacement	Used in
Baseline	`no`	`lru`	All tasks
IP-stride	`ip_stride`	`lru`	Task 1
SPP	`spp`	`lru`	Tasks 1, 3
RRIP	`no`	`rrip`	Task 2
I-RRIP	`no`	`irrip`	Task 2A
Dyn. I-RRIP	`no`	`dyn_irrip`	Tasks 2B, 3
SPP + Dyn. I-RRIP	`spp`	`dyn_irrip`	Task 3

1.3 Metrics

$$\text{Speedup} = \frac{\text{IPC with optimisation}}{\text{IPC of baseline (no prefetching)}} \tag{1}$$

$$\text{L1D Coverage} = \frac{\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses}_{\text{prefetcher}}}{\text{L1D load misses}_{\text{baseline}}} \tag{2}$$

$$\text{Prefetch Accuracy} = \frac{\text{useful prefetches}}{\text{prefetches issued to the lower level}} \tag{3}$$

$$\text{MPKI} = \frac{\text{load misses}}{\text{instructions}/1000} \tag{4}$$

MPKI and prefetch accuracy are taken directly from the ChampSim statistics. ChampSim's accuracy (Equation (3)) divides the prefetched lines that a demand load later hit (`pf_useful`) by the prefetches that actually fetched a line from L2 (`pf_lower_level`); prefetches that were merged in the prefetch queue or found the line already in the L1D do not count. A prefetch is *late* when a demand load arrives while it is still in the MSHR; such a load still counts as an L1D miss. Averages across traces are geometric means for speedup and arithmetic means for MPKI, accuracy and coverage.

Workflow. Every version was built and run with `artifacts/run_spp.py`, which snapshots the prefetcher source to `SPP_prefetchers/<name>.l1d_pref`, builds it, logs a note in `runsTrackers.txt` and stores the outputs in `results_pf<trace>/<name>.txt`. `artifacts/parse_results.py` turns all outputs into `results_summary.csv`, and `artifacts/plot_results.py` draws the figures of this report.

2. Task 1: Data Prefetching

2.1 Task 1A: Understanding the IP-Stride Prefetcher

`prefetcher/ip_stride.l1d_pref` is invoked by the L1D on every demand *load* (hit or miss; stores and prefetches do not train it) with the virtual address and the IP of the load.

Tracker table. A fully associative table of 24 `IP_TRACKER` entries, each holding the IP tag, the last cache-line address, the last stride and an LRU counter. Only the low 8 bits of the IP are kept (`MASK_IP = 0xFF`), so loads whose IPs differ by a multiple of 256 alias to the same entry; the cheap tag is traded for occasional pollution. On a miss in the table the LRU entry is replaced, its address recorded, and nothing is prefetched.

Training. Addresses are converted to cache-line granularity (`addr >> 6`), so accesses within the same 64 B line give a stride of 0 and are ignored. The signed stride to the previous line of the same IP is computed with an explicit sign (the variables are unsigned), then the entry's last address and last stride are updated and the entry becomes MRU.

Prefetch issue. A stride is trusted only when it equals the previous stride of that IP, i.e. prefetching starts on the third access of a constant-stride stream; there is no confidence counter, so a single irregular access costs two prefetch opportunities. The prefetcher then issues `PREFETCH_DEGREE = 3` lines at $+s, +2s, +3s$, stopping at the first one outside the current 4 KB page (virtual pages are not contiguous in physical memory). Lines are filled into the L1D while the L1D MSHR is less than half full and only into the L2 otherwise, so prefetches never starve demand misses of MSHR entries. `prefetch_line` drops the request silently when the 16-entry prefetch queue is full.

Limitations. One stride per IP cannot describe a repeating sequence such as $(+1, +2, +2, +2, +3)^n$: the stride check fails at every change. It also confirms on two equal strides only, holds just 24 IPs, and prefetches at most three lines ahead, which is late for streams with long memory latency. These limitations motivate the design below.

2.2 Task 1B: Stride Pattern Prefetcher (SPP)

2.2.1 Final Design (`spp16`)

Table 3 lists the state. Every tracked IP owns two small pattern tables that map a *history* of recent strides to the stride that followed it: a short table whose history is the last stride ($K_s = 1$) and a long table whose history (the *signature*) is the last three strides ($K_\ell = 3$). A constant stride s is simply the short-table entry $s \to s$, so the short table covers uniform strides (and two-stride alternations), while the long table separates non-uniform sequences such as $(+1, +2, +2, +2, +3)^n$, in which the stride after $+2$ depends on how many $+2$s came before.

Table 3. State of the final SPP (`spp16`).

Structure	Entries	Contents
IP tracker	128	8-bit IP tag, last line address, last 3 strides, strides-seen (0–3), 2-bit chooser, LRU
Short table (per IP)	8	history = 1 stride → predicted next stride, valid, LRU
Long table (per IP)	8	signature = 3 strides → predicted next stride, valid, LRU
Throttling	1	window counters (fetched, useful), current degree

Per-access algorithm. On every L1D demand load from a tracked IP with a non-zero stride s:

1. **Grade.** Look up the entries matching the *old* history in both tables; each predicted this access and is correct if its stored stride equals s. A missing entry counts as wrong.
2. **Choose.** Update the per-IP 2-bit chooser: $+1$ if only the long table was right, -1 if only the short table was right. New IPs start at 0 (short table) so a constant stride is predicted from the third access, exactly as fast as IP-stride.
3. **Train** both tables with (old history → s): overwrite the prediction of a matching entry or allocate the LRU entry. The long table trains only once three strides have been seen (no zero-padded histories).
4. **Shift** s into the history.
5. **Throttle.** If the current window is complete, recompute the degree (Section 2.2.3).
6. **Walk.** Use the long table if chooser ≥ 2, else the short table; if it has no entry for the current history the other table is tried (fallback). Starting from the current line, repeatedly add the predicted stride, issue a prefetch, and shift the predicted stride into a *copy* of the history, for up to *degree* steps or until the address leaves the 4 KB page or no entry matches. Fill level follows the IP-stride rule (L1D if the MSHR is less than half full, else L2).

The chooser makes the non-uniform mode a reversible, per-IP fall-back: the uniform (short) predictor has priority, and an IP moves to the signature only while the signature predicts it better. Table 4 maps the requirements of the assignment to the design.

Core of `l1d_prefetcher_operate` (`prefetcher/spp.l1d_pref`, abridged)

```
// 1. grade both predictors (entries that predicted THIS access)
bool long_ready = (t.strides_seen >= K_LONG);
TableEntry* entry_short = matchHistory(t.shortTable, t.lastKstrides);
TableEntry* entry_long  = long_ready ? matchHistory(t.longTable, t.lastKstrides) :
```

Table 4. Assignment requirements and how the final SPP meets them.

Requirement	Implementation
Per-IP pattern table	128-entry IP tracker, each with its own short and long table
Uniform vs. non-uniform	short table ($s \to s$) vs. 3-stride signature table
Uniform first, fall back	chooser starts at short; moves to the signature only when it predicts better, and back
Signature $\to$ next stride	long table maps the last 3 strides to the next stride
Degree > 1	lookahead walk of up to 8 lines along the predicted path
Accuracy-based throttling	degree 8/4/2 from windowed accuracy (Section 2.2.3)
Page boundary	every candidate is checked against the page of the trigger access

```
    nullptr;
bool short_correct = entry_short && entry_short->pred_stride == stride;
bool long_correct  = entry_long  && entry_long->pred_stride  == stride;
// 2. tournament chooser
if (long_correct && !short_correct)      { if (t.chooser < CHOOSER_MAX) t.chooser++; }
else if (short_correct && !long_correct) { if (t.chooser > 0) t.chooser--; }
// 3. train both tables, 4. shift history
trainTable(t.shortTable, entry_short, t.lastKstrides, stride);
if (long_ready) trainTable(t.longTable, entry_long, t.lastKstrides, stride);
ShiftStride(t.lastKstrides, stride, t.lastKstrides);
// 5. accuracy-based throttling (sets cur_degree)
updateThrottle(pf_useful, pf_lower_level);
// 6. lookahead walk along the predicted path
bool use_long = (t.chooser >= CHOOSER_THRESHOLD) && (t.strides_seen >= K_LONG);
Table &chosen = use_long ? t.longTable : t.shortTable;
Table &other  = use_long ? t.shortTable : t.longTable;
int64_t pattern[K_LONG];  copy(t.lastKstrides, pattern);
uint64_t line = cl_addr;
for (int p = 0; p < cur_degree; ++p) {
    TableEntry* e = matchHistory(chosen, pattern);
    if (!e && USE_FALLBACK) e = matchHistory(other, pattern);
    if (!e) break;
    uint64_t pf_addr = (line + e->pred_stride) << LOG2_BLOCK_SIZE;
    line = pf_addr >> LOG2_BLOCK_SIZE;
    if ((pf_addr >> LOG2_PAGE_SIZE) != (addr >> LOG2_PAGE_SIZE)) break;
    prefetch_line(ip, addr, pf_addr, MSHR.occupancy < MSHR.SIZE/2 ? FILL_L1 : FILL_L2,
    0);
    ShiftStride(pattern, e->pred_stride, pattern);
}
```

2.2.2 How the Final Design Was Reached

Each version was run on all three traces (spp1 and spp5 on one trace only). Table 5 summarises the path; the complete statistics are in Table 13.

STEP 1 – LOOKAHEAD (SPP1 $\to$ SPP2). The first version kept, per IP, a table from the last three strides to the next one and issued one prefetch. Degree 1 is too late (27% of useful prefetches late on Trace 1); walking the predicted path four steps ahead (each step adding the predicted stride to the previous *predicted* line, not to the trigger) roughly doubled coverage (2.3% $\to$ 4.1%) and halved lateness.

Table 5. Design history (speedup over the no-prefetch baseline). Storage for spp14–16 from Section 2.2.4.

Version	Change	T1	T2	T3	Geomean
ip_stride	reference (24 IPs, degree 3)	1.0044	1.0007	1.0021	1.0024
spp1	per-IP pattern table, $K = 3$, degree 1	1.0020	–	–	–
spp2	lookahead walk, degree 4	1.0033	1.0011	1.0029	1.0024
spp3	spp2 with $K = 1$	1.0052	1.0007	1.0043	1.0034
spp4	escalating dynamic K, 2-bit confidence, off switch	1.0049	1.0007	1.0035	1.0030
spp5	spp4, 64 trackers	–	1.0049	–	–
spp6	spp4, 256 trackers	1.0007	1.0026	1.0001	1.0011
spp7	spp4, 64 trackers, degree 8	1.0058	1.0060	1.0100	1.0073
spp8	spp3, 256 trackers, degree 8	1.0070	1.0147	1.0179	1.0132
spp9	spp8 with $K = 3$, 64 entries/IP	1.0060	1.0155	1.0095	1.0103
spp10	tournament $K = 1$ / $K = 3$, fallback on	1.0073	1.0148	1.0183	1.0134
spp11	spp10, fallback off	1.0073	1.0148	1.0177	1.0133
spp12	+ throttling, degrees 16/12/8/4/2	1.0065	1.0149	1.0204	1.0139
spp14	throttling capped at 8 (8/4/2)	1.0079	1.0157	1.0196	**1.0144**
spp15	spp14, 128 trackers, 8/16 entries (12 KB)	1.0081	1.0141	1.0165	1.0129
spp16	**spp15, long table 8 entries (7.8 KB)**	**1.0080**	**1.0141**	**1.0165**	**1.0129**

STEP 2 – LONGER HISTORY IS NOT BETTER (SPP2 VS. SPP3). With only K changed, $K = 1$ beat $K = 3$ on coverage on every trace (7.1 vs. 4.1% on Trace 1, 6.2 vs. 3.3% on Trace 3) while $K = 3$ was more accurate (55 vs. 48%, 57 vs. 33% on Trace 2). A three-stride history is a narrower key: it warms up two strides later, one irregular stride spoils the next three histories, and the walk stops at the first unseen signature. The predictable misses in these traces are mostly uniform or short patterns.

STEP 3 – ESCALATING K FAILS (SPP4–SPP7, BRIEF). We made K dynamic per IP: start at 1, raise K when an entry's error counter saturates, switch the IP off at maximum K. It never beat fixed $K = 1$, and with 256 trackers (spp6) it collapsed (geomean 1.0011, 2,460 lines fetched on Trace 1 instead of ≈40,000). Out-of-order load reordering and loop restarts look like conflicts, so IPs escalated, cleared their tables and never came back down; with 24 trackers the frequent tracker evictions had been resetting them and hiding the problem. Lesson: any change of history length must be reversible and must not discard trained state.

STEP 4 – CAPACITY AND TIMELINESS (SPP8, SPP9). Two findings from step 3 carried over: more trackers help Trace 2, whose large code footprint (L1I MPKI 40) means many live load IPs (24 → 64 trackers: 0.8 → 1.9% coverage), and degree 8 removes most late prefetches (Trace 3: 32% → 11% late). Applying both to the simple $K = 1$ design gave spp8, the first large jump (geomean 1.0024 → 1.0132; Trace 3 coverage 17.1%). The same configuration with $K = 3$ (spp9) was worse overall (1.0103) but *better* on Trace 2 (1.0155 vs. 1.0147, accuracy 71 vs. 42%): no single history length is best for every workload.

STEP 5 – TOURNAMENT CHOOSER (SPP10, SPP11). To get $K = 1$ where it wins and $K = 3$ where it wins, both tables are kept and trained, and a per-IP 2-bit counter chooses, graded on every access. Table 6 shows the behaviour with the large tables. spp10 reached the best coverage on every trace (12.2/5.1/17.6%) and a geomean of 1.0134, never worse than spp8. On a synthetic $(+1, +2, +2, +2, +3)^n$ IP it prefetched 96.9% of accesses versus 76.3% for $K = 1$ alone, while matching $K = 1$ on uniform streams. It did not reproduce spp9's precision on Trace 2: there both predictors are right for only ≈11% of next strides, so spp9's advantage came from *not* prefetching when its

signature was unseen. Turning the fallback off (spp11) recovered almost none of it (accuracy 45 → 47%, geomean 1.0133), so the fallback was kept and the remaining waste was attacked with throttling.

Table 6. Chooser statistics of spp14 (24/64 entries per IP); counters include warm-up.

Statistic	Trace 1	Trace 2	Trace 3
Walks driven by the long table	12.1%	30.7%	33.6%
Next stride predicted correctly: short / long	22.0 / 18.3%	10.8 / 10.9%	58.2 / 57.0%
Chooser flips (each direction)	≈15.4k	≈17.6k	≈29.9k

Step 6 – accuracy throttling (spp12 → spp14). Detailed in Section 2.2.3: throttling *down* in inaccurate phases helped Trace 3 strongly, throttling *up* beyond 8 did not help anywhere, so the final bands cap the degree at 8 (geomean 1.0144).

Step 7 – storage (spp14 → spp16). spp14 needs ≈88 KB, more than the 48 KB L1D. Halving the trackers to 128 and shrinking the per-IP tables to 8/16 entries (spp15, 12 KB) cost 0.15 points of geomean, mostly on Trace 3 (1.0196 → 1.0165) and Trace 2 (1.0157 → 1.0141), while Trace 1 was unchanged. Shrinking the long table further to 8 entries (spp16, 7.8 KB) changed nothing measurable (geomean 1.0129 in both). With small tables the long table drives only 2.5/0.7/2.0% of walks, so most of the benefit comes from the short predictor, lookahead and throttling. spp16 is the final design: about 90% of spp14's gain for 9% of its storage.

2.2.3 Accuracy-Based Throttling

The prefetcher reads the L1D's own counters `pf_useful` and `pf_lower_level`. A *window* ends every 256 prefetches that fetched a line from L2; its accuracy is Δ`pf_useful`/Δ`pf_lower_level` over the window, and it sets the degree used by all walks until the next window ends (Table 7). The first window uses degree 8. ChampSim resets these counters at the end of warm-up; a counter that goes backwards simply starts a new window. In hardware the same result needs only two small window counters (Table 9).

Table 7. Prefetch degree chosen for each window-accuracy range (final design).

Window accuracy	Degree	Reason
acc $\geq$ 30%	8	distance needed for timeliness; deeper walks only add redundant or page-cut requests
15% $\leq$ acc $<$ 30%	4	poor phase: halve the walk, keep the most accurate first steps
acc $<$ 15%	2	bad phase: keep sampling (degree 0 could never measure recovery)

How the thresholds were chosen. The data showed that low accuracy alone is not a reason to throttle: Trace 3 runs at only 22–24% accuracy at degree 8, yet going from degree 4 to 8 was its largest single gain (spp4 1.0035 → spp7 1.0100) because lateness fell from 32% to 11%. A threshold much above 30% would therefore throttle Trace 3's useful phases. At 30% the average window accuracy (50/65/41% on the three traces) stays above the threshold, so the common case keeps degree 8, and only clearly worse phases are cut. Below 15%, more than 85% of fetched lines are never used, which is pure pollution of the 48 KB L1D, so the degree is reduced again. The window of 256 fetched prefetches gives 470–950 decisions per run; at 50% accuracy its sampling error is about $\pm$3%, small next to the 15-point band width.

WHY THESE DEGREES. spp12 used five bands up to degree 16 ($\geq$60% $\rightarrow$ 16, 40–60% $\rightarrow$ 12, 20–40% $\rightarrow$ 8, 10–20% $\rightarrow$ 4, $<$10% $\rightarrow$ 2). Table 8 compares it with the final bands. On Trace 2, degree 16 was used in 51% of windows, raised requests from 2.06M to 2.49M, but fetched and useful lines did not change: the extra steps were already cached, merged in the prefetch queue, or cut at the page boundary. On Trace 1, the 57% of windows at degree 12–16 added 5.7k fetched but only 0.8k useful lines (speedup 1.0073 $\rightarrow$ 1.0065). Capping at 8 (spp14) fixed Trace 1 (1.0079), improved Trace 2 (1.0157) and kept almost all of Trace 3's gain (1.0196 vs. 1.0204), with the highest accuracy of all versions. Degrees 4 and 2 halve the walk at each lower band, which keeps the most accurate first steps while still producing fetches from which the next window can measure recovery.

Table 8. Effect of the throttling bands (share of windows at each degree; speedup; accuracy).

Version	Trace	deg 2	deg 4	deg 8	deg 12	deg 16	Speedup	Accuracy
spp10 (fixed 8)	1	–	–	100%	–	–	1.0073	44.5%
spp12 (to 16)	1	0.8%	4.6%	37.7%	33.4%	23.5%	1.0065	43.1%
spp14 (to 8)	1	1.4%	13.6%	85.0%	–	–	1.0079	46.6%
spp10 (fixed 8)	2	–	–	100%	–	–	1.0148	45.3%
spp12 (to 16)	2	1.3%	5.4%	27.7%	14.4%	51.2%	1.0149	45.2%
spp14 (to 8)	2	1.6%	15.6%	82.8%	–	–	1.0157	50.2%
spp10 (fixed 8)	3	–	–	100%	–	–	1.0183	24.0%
spp12 (to 16)	3	7.5%	15.6%	39.3%	17.8%	19.8%	1.0204	33.5%
spp14 (to 8)	3	10.5%	26.8%	62.7%	–	–	1.0196	35.2%

2.2.4 Storage Overhead

Table 9 counts the bits a hardware implementation of spp16 needs (the simulator uses 64-bit C++ variables for convenience). Strides are stored in 7 bits ($-64 \ldots +63$ lines): a prefetch can never leave the 64-line page, so larger strides can be represented by one saturated "out of range" code without losing a prefetch. Replacement uses true LRU ($\log_2$ of the table size per entry).

FURTHER REDUCTION. Half of the remaining bits are the long tables and the stored signatures. A single shared long table, direct-mapped by a hash of the three-stride signature (e.g. 1024 entries of a 6-bit partial tag, a 7-bit prediction and a valid bit, $\approx$14 kbit), would replace the per-IP long tables and their LRU bits and bring the design to roughly 5.5 KB; IPs with the same pattern would also share training. Storing the last address as a page hash plus a 6-bit offset would save a further $\approx$5 kbit.

2.2.5 Results

Table 11 compares the baseline, the provided IP-stride prefetcher and the final SPP; Figures 1 to 4 plot the same data.

2.2.6 Discussion

WHY THE ABSOLUTE GAINS ARE SMALL. Table 12 shows the baseline bottlenecks. Traces 1 and 2 are dominated by instruction misses (L1I MPKI 14.5 and 40.0, and more L2 instruction than data misses) and Trace 1 also by branch mispredictions (16.1 MPKI); an L1D prefetcher cannot touch these stalls. Trace 3 is the most data-bound, but its L1D misses only 3.4 times per kilo-instruction. Speedups of 1–2% are therefore close to what an L1D prefetcher can deliver here; relative to IP-stride the final SPP gains 5.4$\times$ more.

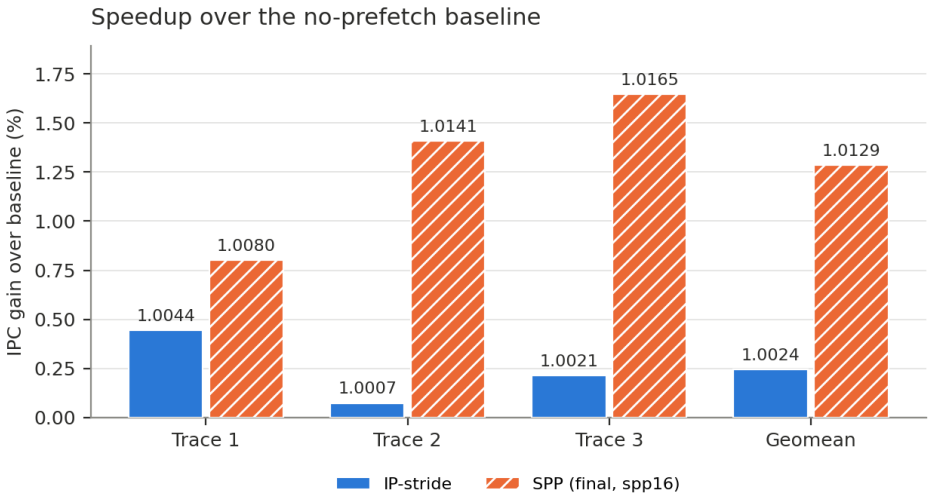


Figure 1. Speedup over the no-prefetch baseline (bars: IPC gain in %; labels: speedup).

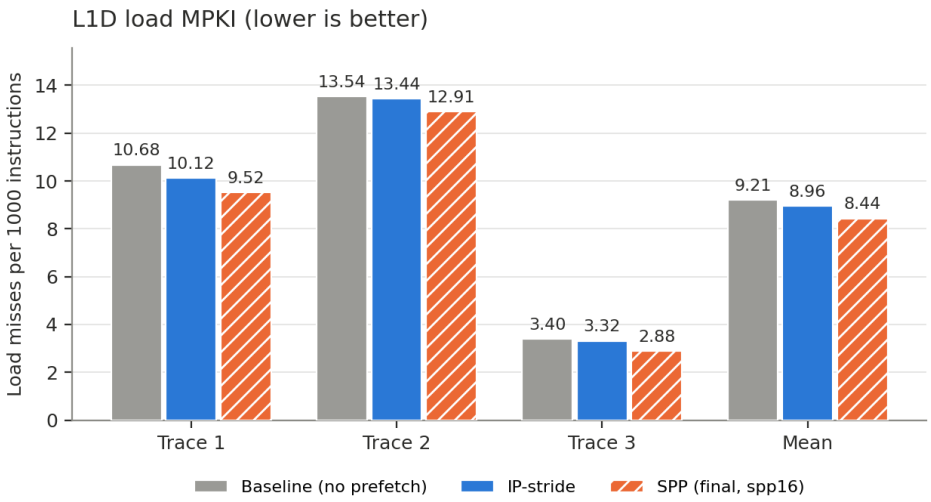


Figure 2. L1D load MPKI.

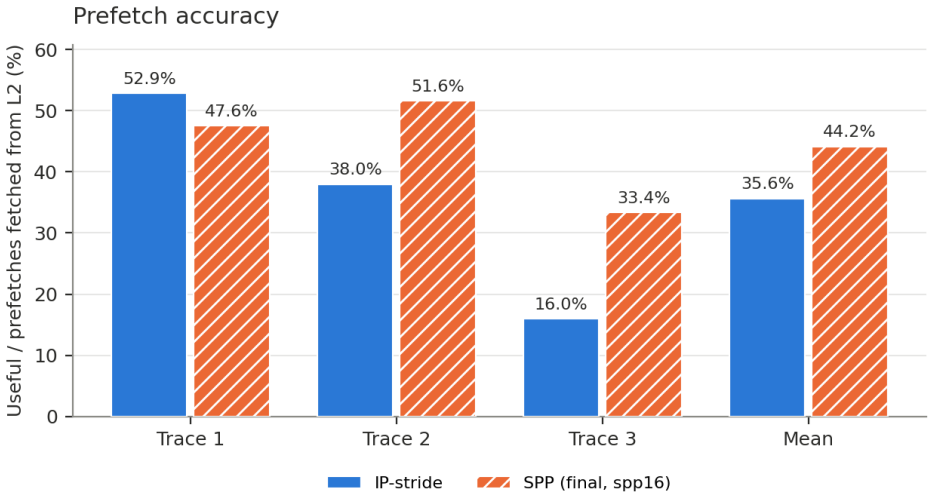


Figure 3. Prefetch accuracy (useful / prefetches fetched from L2).

Table 9. Bit-level storage breakdown of the final SPP (`spp16`).

Structure	Field	Bits/entry	Entries	Total bits
IP tracker	Valid	1	128	128
	IP tag (`MASK_IP 0xFF`)	8	128	1,024
	Last line address	58	128	7,424
	Stride history (3×7)	21	128	2,688
	Strides seen (0–3)	2	128	256
	Chooser	2	128	256
	LRU	7	128	896
Short table	History (1 stride)	7	1,024	7,168
(8 per IP)	Predicted stride	7	1,024	7,168
	Valid	1	1,024	1,024
	LRU	3	1,024	3,072
Long table	Signature (3 strides)	21	1,024	21,504
(8 per IP)	Predicted stride	7	1,024	7,168
	Valid	1	1,024	1,024
	LRU	3	1,024	3,072
Throttling	Window fetched counter	9	1	9
	Window useful counter	10	1	10
	Current degree	4	1	4
Total				**63,895 bits (7,987 B $\approx$ 7.8 KB)**

Table 10. Storage versus performance (same algorithm, different sizes).

Version	Trackers	Short / long entries	Bits	Storage	Geomean
spp14	256	24 / 64	721,943	88.1 KB	1.0144
spp15	128	8 / 16	98,711	12.0 KB	1.0129
spp16	**128**	**8 / 8**	**63,895**	**7.8 KB**	**1.0129**

Trace by trace. *Trace 1*: SPP doubles IP-stride's coverage (10.9 vs. 5.2%) at slightly lower accuracy; its predictable misses are mostly regular streams, which the deeper walk and 128 trackers serve better. *Trace 2*: many live IPs; IP-stride's 24 trackers thrash (0.7% coverage), whereas SPP covers 4.6% with higher accuracy (51.6%), giving the largest relative gain (about 19$\times$ IP-stride's speedup). *Trace 3*: regular streams whose prefetches were mostly late with IP-stride (53% late); the deeper walk cuts lateness to 18%, and throttling raises accuracy from 16% to 33%, giving the best result (1.0165, coverage 15.3%).

Insights. (i) Timeliness mattered more than prediction quality: the largest single gains came from walking further ahead. (ii) For these traces a short history covers most predictable misses; longer signatures are more precise but cover less, so they must be a reversible fall-back, never a one-way escalation. (iii) Throttling helps by *reducing* the degree in bad phases; beyond degree 8 extra lookahead is wasted inside a 4 KB page. (iv) Request redundancy remains: over 95% of requests do not fetch a line (2.2M requested vs. 97k fetched on Trace 1), mostly re-requests of lines already

Table 11. Task 1 results (L1D prefetching, LRU at L2). SPP = final design `spp16`.

Trace	Config	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace 1	Baseline	0.2320	1.0000	10.68	–	–
	IP-stride	0.2330	1.0044	10.12	52.9	5.2
	SPP	0.2339	1.0080	9.52	47.6	10.9
Trace 2	Baseline	0.2506	1.0000	13.54	–	–
	IP-stride	0.2508	1.0007	13.44	38.0	0.7
	SPP	0.2541	1.0141	12.91	51.6	4.6
Trace 3	Baseline	0.3819	1.0000	3.40	–	–
	IP-stride	0.3827	1.0021	3.32	16.0	2.5
	SPP	0.3882	1.0165	2.88	33.4	15.3
Mean	IP-stride	–	1.0024	8.96	35.6	2.8
	SPP	–	**1.0129**	**8.44**	**44.2**	**10.3**

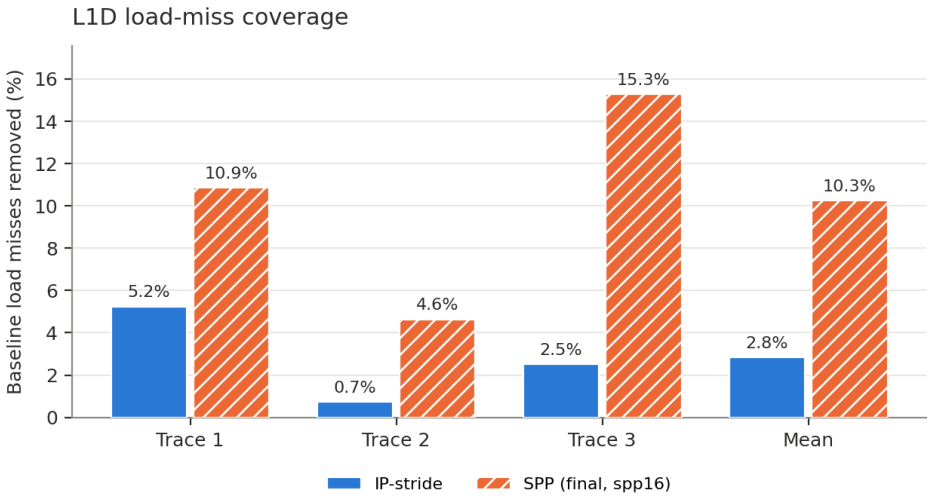


Figure 4. L1D load-miss coverage.

Table 12. Baseline bottlenecks (no prefetching).

Trace	IPC	L1I MPKI	L1D load MPKI	L2 instr. MPKI	L2 data MPKI	Branch MPKI
Trace 1	0.232	14.5	10.7	10.4	7.3	16.1
Trace 2	0.251	40.0	13.5	19.6	8.6	6.3
Trace 3	0.382	2.1	3.4	1.1	2.8	7.2

covered by the previous walk; a per-IP "already prefetched up to step n" marker would remove them without changing coverage.

2.2.7 Full Comparison of All Versions

Table 13. Full comparison of all evaluated versions (spp13 omitted: identical to spp14 up to rounding). Coverage, accuracy and lateness are percentages; Fetched = prefetches that brought a line from L2; Requested = calls to `prefetch_line`.

Version	Trace	IPC	Speedup	L1D ld MPKI	Cov. (%)	Acc. (%)	Late (%)	Fetched	Useful	Requested
baseline	1	0.2320	1.0000	10.68	–	–	–	0	0	0
baseline	2	0.2506	1.0000	13.54	–	–	–	0	0	0
baseline	3	0.3819	1.0000	3.40	–	–	–	0	0	0
ip_stride	1	0.2330	1.0044	10.12	5.2	52.9	14.7	30971	16378	298580
ip_stride	2	0.2508	1.0007	13.44	0.7	38.0	6.0	11521	4380	152145
ip_stride	3	0.3827	1.0021	3.32	2.5	16.0	52.7	17069	2731	200869
spp1	1	0.2325	1.0020	10.43	2.3	60.7	27.2	10569	6412	76900
spp2	1	0.2328	1.0033	10.24	4.1	55.4	14.1	22038	12213	289766
spp2	2	0.2508	1.0011	13.46	0.6	57.2	5.2	4951	2834	89619
spp2	3	0.3830	1.0029	3.29	3.3	22.1	36.8	14868	3293	180162
spp3	1	0.2332	1.0052	9.92	7.1	48.5	11.5	46786	22670	616360
spp3	2	0.2507	1.0007	13.43	0.8	32.9	5.0	16517	5438	249386
spp3	3	0.3835	1.0043	3.19	6.2	18.3	35.3	35011	6412	548178
spp4	1	0.2331	1.0049	10.01	6.3	50.3	11.1	39861	20060	507874
spp4	2	0.2508	1.0007	13.43	0.8	33.5	5.0	15812	5301	239275
spp4	3	0.3832	1.0035	3.26	4.2	19.5	31.7	22793	4453	437681
spp5	2	0.2518	1.0049	13.28	1.9	43.9	2.6	21355	9381	537738
spp6	1	0.2322	1.0007	10.62	0.6	78.2	7.8	2460	1924	38650
spp6	2	0.2512	1.0026	13.48	0.4	75.9	5.1	2419	1837	112777
spp6	3	0.3819	1.0001	3.40	0.1	60.9	18.8	184	112	33219
spp7	1	0.2334	1.0058	9.70	9.2	45.6	5.4	77930	35559	1522111
spp7	2	0.2521	1.0060	13.22	2.4	34.8	1.5	36327	12641	935257
spp7	3	0.3857	1.0100	3.08	9.5	23.2	11.0	44226	10271	964729
spp8	1	0.2336	1.0070	9.41	11.9	43.5	4.4	125677	54630	3126858
spp8	2	0.2543	1.0147	12.88	4.9	42.2	1.4	63014	26612	2049660
spp8	3	0.3887	1.0179	2.82	17.1	21.8	14.5	83769	18243	2403452
spp9	1	0.2334	1.0060	9.81	8.1	56.5	5.5	55727	31500	1748725
spp9	2	0.2545	1.0155	12.85	5.0	71.3	1.7	29353	20942	1371169
spp9	3	0.3855	1.0095	3.07	9.7	27.1	18.4	34760	9423	1709591
spp10	1	0.2337	1.0073	9.37	12.2	44.5	4.4	124230	55233	3065988
spp10	2	0.2543	1.0148	12.85	5.1	45.3	1.4	64157	29037	2057559
spp10	3	0.3889	1.0183	2.80	17.6	24.0	14.8	75690	18144	2223233
spp11	1	0.2337	1.0073	9.40	12.0	45.0	4.5	119368	53732	2952185
spp11	2	0.2543	1.0148	12.85	5.0	47.2	1.6	58896	27819	1951350
spp11	3	0.3887	1.0177	2.84	16.6	23.7	13.9	71822	17041	2119438
spp12	1	0.2335	1.0065	9.38	12.2	43.1	4.3	129922	56054	3504498
spp12	2	0.2543	1.0149	12.84	5.1	45.2	1.4	64064	28950	2492034
spp12	3	0.3897	1.0204	2.77	18.6	33.5	15.3	53851	18063	1480879
spp14	1	0.2339	1.0079	9.39	12.0	46.6	4.9	114478	53306	2748438
spp14	2	0.2545	1.0157	12.82	5.3	50.2	1.6	56174	28206	1829211
spp14	3	0.3894	1.0196	2.80	17.8	35.2	16.5	48678	17123	1334309
spp15	1	0.2339	1.0081	9.52	10.9	47.5	5.4	97277	46244	2206887
spp15	2	0.2541	1.0141	12.91	4.7	52.2	2.1	38336	19996	1169160
spp15	3	0.3882	1.0165	2.89	15.2	33.6	18.3	43637	14662	957902

continued on next page

Table 13 (continued)

Version	Trace	IPC	Speedup	L1D ld MPKI	Cov. (%)	Acc. (%)	Late (%)	Fetched	Useful	Requested
spp16	1	0.2339	1.0080	9.52	10.9	47.6	5.2	97421	46334	2200401
spp16	2	0.2541	1.0141	12.91	4.6	51.6	2.0	38673	19964	1194763
spp16	3	0.3882	1.0165	2.88	15.3	33.4	17.8	44075	14707	907982

3. Task 2: Cache Replacement Policy

3.1 Baseline: SRRIP at the L2

The provided `replacement/rrip.l2c_repl` implements SRRIP with a 2-bit re-reference prediction value (RRPV, maxRRPV = 3):

- **Insertion:** a newly filled line gets RRPV = 2 (long re-reference interval).
- **Promotion:** a hit sets RRPV = 0 (near-immediate re-reference).
- **Eviction:** the first way with RRPV = 3 is chosen; if none exists, all RRPVs in the set are incremented and the search repeats.

[Verify against the provided file.]

3.2 Task 2A: Instruction-Conscious RRIP (I-RRIP)

3.2.1 Interface Change

[Describe how `update_replacement_state` was extended with an `isInstruction` argument: the declaration in `inc/cache.h`, every call site in `src/cache.cc`, and how the value is taken from `packet.instruction`.]

3.2.2 Policy Design

Table 14. Insertion, promotion and eviction in SRRIP and I-RRIP.

Part	SRRIP	I-RRIP
Insertion	RRPV = 2 for every line	*[instruction lines: . . . ; data lines: . . .]*
Promotion	RRPV = 0 on any hit	*[. . .]*
Eviction	First line with RRPV = 3; age set otherwise	*[. . .]*

Insertion. *[Rationale for modifying or not modifying insertion.]*

Promotion. *[Rationale for modifying or not modifying promotion.]*

Eviction. *[Rationale for modifying or not modifying eviction.]*

> **Why this design is the best of those we tried.** *[Deliverable: defend the choice. List the alternatives evaluated and the evidence against each.]*

3.2.3 Results

Discussion. *[Change in L2 instruction MPKI vs. data MPKI per trace, and the resulting IPC change.]*

Table 15. Task 2A results (no prefetcher).

Trace	L2 policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
	LRU	–	1.000	–	–	–
Trace 1	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–
	LRU	–	1.000	–	–	–
Trace 2	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–
	LRU	–	1.000	–	–	–
Trace 3	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–



Figure 5. Task 2A: speedup of SRRIP and I-RRIP.



Figure 6. Task 2A: L2C, L2 instruction and L2 data load MPKI.

3.3 Task 2B: Dynamic I-RRIP

3.3.1 Set-Dueling Design

[Leader sets (how many, how chosen), what is counted in them, how often the counters are evaluated / reset, how follower sets pick a policy.]

The decision metric is

$$r = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}}, \qquad \text{policy} = \begin{cases} \text{I-RRIP} & r > \theta \\ \text{SRRIP} & \text{otherwise} \end{cases} \tag{5}$$

with threshold $\theta = -$. *[Edit if you changed the formula or added parameters.]*

3.3.2 Rationale for the Leader Sets and Decision Logic

Number of leader sets. *[Deliverable.]*

Decision logic. *[Deliverable.]*

3.3.3 Sensitivity Study

Table 16. Sensitivity of Dynamic I-RRIP to the threshold θ (speedup over LRU).

θ	Trace 1	Trace 2	Trace 3	Geomean	Mean L2 instr. MPKI
–	–	–	–	–	–
–	–	–	–	–	–
–	–	–	–	–	–
–	–	–	–	–	–
–	–	–	–	–	–

Plot: `figures/task2b_sensitivity.pdf` / .png

Figure 7. Speedup of Dynamic I-RRIP against the threshold θ.

3.3.4 Results

Table 17. Task 2B results (no prefetcher, $\theta = -$).

Trace	L2 policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace 1	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–
	Dyn. I-RRIP	–	–	–	–	–
Trace 2	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–
	Dyn. I-RRIP	–	–	–	–	–
Trace 3	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–
	Dyn. I-RRIP	–	–	–	–	–

Discussion. *[Fraction of time each trace spends in I-RRIP mode, and where dynamic switching recovers data misses that static I-RRIP lost.]*



Figure 8. Task 2B: speedup of SRRIP, I-RRIP and Dynamic I-RRIP.



Figure 9. Task 2B: L2C, L2 instruction and L2 data load MPKI.

4. Task 3: SPP with Prefetch-Aware Dynamic I-RRIP

4.1 Design

[How `dyn_irrip.l2c_repl` treats lines whose packet type is `PREFETCH`: insertion, first demand hit, promotion, eviction priority, set-dueling counters.]

Table 18. Treatment of each line class in the prefetch-aware policy.

Line class	Insertion	Promotion	Eviction priority
Instruction (demand)	*[. . .]*	*[. . .]*	*[. . .]*
Data (demand)	*[. . .]*	*[. . .]*	*[. . .]*
Prefetched	*[. . .]*	*[. . .]*	*[. . .]*

4.2 Rationale for Handling Prefetched Lines

[Deliverable.]

4.3 Results

Discussion. *[Show that the combination outperforms both standalone SPP and Dynamic I-RRIP in IPC, and explain where the extra gain comes from.]*

Table 19. Task 3 results: performance and prefetch metrics.

Trace	Config	IPC	Speedup	Accuracy (%)	Coverage (%)	L1D MPKI
Trace 1	SPP	0.2339	1.0080	47.6	10.9	9.52
	Dyn. I-RRIP	–	–	–	–	–
	SPP + Dyn. I-RRIP	–	–	–	–	–
Trace 2	SPP	0.2541	1.0141	51.6	4.6	12.91
	Dyn. I-RRIP	–	–	–	–	–
	SPP + Dyn. I-RRIP	–	–	–	–	–
Trace 3	SPP	0.3882	1.0165	33.4	15.3	2.88
	Dyn. I-RRIP	–	–	–	–	–
	SPP + Dyn. I-RRIP	–	–	–	–	–

Table 20. Task 3 results: L2 load MPKI.

Trace	Config	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace 1	SPP	16.75	10.41	6.35
	Dyn. I-RRIP	–	–	–
	SPP + Dyn. I-RRIP	–	–	–
Trace 2	SPP	27.70	19.66	8.04
	Dyn. I-RRIP	–	–	–
	SPP + Dyn. I-RRIP	–	–	–
Trace 3	SPP	3.34	1.16	2.18
	Dyn. I-RRIP	–	–	–
	SPP + Dyn. I-RRIP	–	–	–

Plot: `figures/task3_speedup.pdf` / .png

Figure 10. Task 3: speedup of SPP, Dynamic I-RRIP and their combination.

Plot: `figures/task3_accuracy_coverage.pdf` / .png

Figure 11. Task 3: prefetch accuracy and coverage.



Figure 12. Task 3: L1D load MPKI.

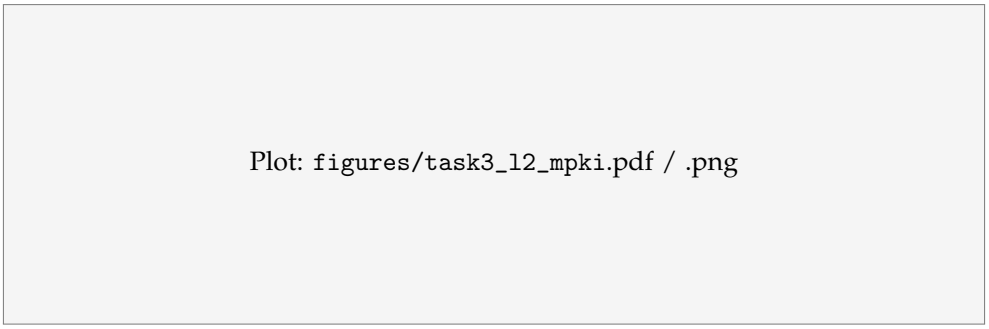


Figure 13. Task 3: L2C, L2 instruction and L2 data load MPKI.

5. Summary

Table 21. Speedup of every configuration over the baseline.

Configuration	Trace 1	Trace 2	Trace 3	Geomean
IP-stride	1.0044	1.0007	1.0021	1.0024
SPP (`spp16`)	1.0080	1.0141	1.0165	1.0129
SRRIP	–	–	–	–
I-RRIP	–	–	–	–
Dyn. I-RRIP	–	–	–	–
SPP + Dyn. I-RRIP	–	–	–	–

[Short closing paragraph: key takeaways from each task.]



Figure 14. Speedup of all configurations.

A. Reproducing the Results

Build, run, parse and plot

```
# baseline and reference prefetcher (once per trace)
./build_champsim.sh no lru baseline
./build_champsim.sh ip_stride lru ip_stride
./bin/<binary> -warmup_instructions 25000000 \
    -simulation_instructions 25000000 -traces ./Trace_Files/Trace1.gz \
    > results_pf1/baseline.txt

# any SPP version: snapshot + build + run on traces 1-3 + log
python3 artifacts/run_spp.py spp16 1 2 3 --jobs 3 --note "final design"

# summary table and figures
python3 artifacts/parse_results.py      # -> results_summary.csv
python3 artifacts/plot_results.py       # -> figures/task1_*.png
```

The source of every evaluated SPP version is kept in `SPP_prefetchers/<name>.l1d_pref`, the raw ChampSim outputs in `results_pf<trace>/<name>.txt`, and one line per run in `runsTrackers.txt`.